

DSA — HASHING NOTES

Python + Striver DSA | Part 1: Array Hashing & Dictionary (Hash Map)

1. What is Hashing?

Core idea: Hashing means storing information about an element so that we can retrieve that information quickly later.

Think of it as: **KEY** → **INFORMATION**

Examples: number → frequency, number → index, character → frequency, value → True/False.

2. Why Do We Need Hashing?

Suppose we have an array and need to know how many times different numbers occur. Scanning the whole array for every query repeats the same work. Hashing lets us preprocess the array once and then answer frequency queries quickly.

Important pattern: Preprocess once → store information → answer many queries quickly.

3. Array Hashing

When the values are small non-negative integers, the number itself can be used as an array index.

Example:

```
arr = [4, 4, 2, 7, 2, 4]
```

```
hash = [0] * 8
```

```
Then for every element x: hash[x] += 1
```

Final table:

Index	0	1	2	3	4	5	6	7
hash[index]	0	0	2	0	3	0	0	1

Meaning: hash[4] = 3 means the number 4 appeared 3 times. The index represents the number; the value stored there represents its information (here, frequency).

4. Array Hashing Code

```
arr = [4, 4, 2, 7, 2, 4]
```

```
hash = [0] * 8
```

```
for x in arr:  
    hash[x] += 1
```

```
print(hash)  
# [0, 0, 2, 0, 3, 0, 0, 1]
```

5. Queries

After building the hash table, a frequency query can be answered directly:

```
query = 4 → hash[4] → 3  
query = 7 → hash[7] → 1
```

```
query = 6 → hash[6] → 0
```

Time: Building the table = $O(N)$. Each query = $O(1)$. For N array elements and Q queries, total work is approximately $O(N + Q)$.

6. Limitation of Array Hashing

Array hashing is not a good choice when values are extremely large, sparse, or inconvenient to use as indexes.

Example: [1000000, 5000000, 1000000000]. Creating an array up to index 1,000,000,000 would waste huge amounts of memory.

Solution: Use a Python dictionary (dict), also called a hash map.

7. Python dict = Hash Map

A dictionary stores pairs in the form **KEY** → **VALUE**.

```
freq = {  
    10: 3,  
    20: 5,  
    30: 2  
}
```

Here, 10, 20, and 30 are keys. Their associated values are 3, 5, and 2.

Important difference: In array hashing, hash[10] means index 10. In a dictionary, freq[10] means the value associated with key 10.

8. Frequency Counting with dict

```
arr = [4, 4, 2, 7, 2, 4]  
  
freq = {}  
  
for x in arr:  
    if x in freq:  
        freq[x] += 1  
    else:  
        freq[x] = 1  
  
print(freq)  
# {4: 3, 2: 2, 7: 1}
```

9. Python Shortcut: dict.get()

The same frequency counting can be written more simply:

```
freq = {}  
  
for x in arr:  
    freq[x] = freq.get(x, 0) + 1
```

How .get(key, default) works:

freq.get(4, 0) → returns the current value for key 4 if it exists.

freq.get(7, 0) → returns 0 if key 7 does not exist.

Therefore, freq.get(x, 0) + 1 means: **get the current frequency of x; if it doesn't exist, start from 0; then add 1.**

10. Array Hashing vs dict

Feature	Array Hashing	dict / Hash Map
Storage idea	Index → information	Key → information
Best for	Small integer range	Large/unpredictable keys
Example	hash[x]	freq[x]
Flexibility	Limited	High
Common DSA use	Simple frequency arrays	Frequency, index, lookup, pairs

11. The Most Important Mental Model

Do not think: “**Hashing means using dict.**”

Think: “**I need to remember some information about an element and retrieve it quickly later.**”

Then ask: **What information do I need to store?**

Examples:

- number → frequency
- number → index
- character → frequency
- number → True/False
- value → another value

12. How to Recognize Hashing in a New Problem

When a problem says or implies things like:

- “How many times does each value occur?”
- “Have we seen this value before?”
- “Find the index where this value occurred.”
- “Check whether a value exists quickly.”
- “Find a pair/value that matches a condition.”

Ask yourself: “**Can I store information about each value so I don't have to repeatedly scan the array?**” If yes, hashing is a strong candidate.

13. Complexity to Remember

For typical Python dict/set operations, lookup, insertion, and deletion are **$O(1)$ average-case**. For array hashing, direct indexing is $O(1)$. Building a frequency table over N elements is $O(N)$.

14. Quick Revision

Hashing: Store information for fast retrieval.

Array hashing: Use the value as an index when the range is small.

Hash map: Use dict when keys are general/large/sparse.

Frequency: key = element, value = count.

Core Python pattern: `freq[x] = freq.get(x, 0) + 1`.

Main DSA idea: preprocess once, then answer lookups quickly.

15. Practice Example

```
arr = [5, 2, 5, 8, 2, 5]
```

```
freq = {}
```

```
for x in arr:
```

```
    freq[x] = freq.get(x, 0) + 1
```

```
print(freq)
```

Expected result: {5: 3, 2: 2, 8: 1}